FH University of Applied Sciences TECHNIKUM WIEN

Artificial Intelligence EF

Modul 3: Maschinelles Lernen

Informatik

Lernergebnisse und Inhalt

Modul 3: „***Verfahren des maschinellen Lernens und des Data Mining einzusetzen, um Analysen und Vorhersagen auf Basis von Echtdaten zu erstellen.***“

1. Überblick Machine Learning
2. Überwachtes Lernen
3. Unüberwachtes Lernen

© FH Technikum Wien

1. Überblick Machine Learning

© FH Technikum Wien

Was ist maschinelles Lernen?

- Teilgebiet der Künstlichen Intelligenz
- Erforschung „intelligenter“ Verfahren, die die menschliche Lernfähigkeit zu imitieren versuchen
- Auch der Versuch, die Funktionsweise des menschlichen Gehirns zu modellieren und verstehen
- Unterschiedliche Formalismen und Vorgehensweisen

© FH Technikum Wien

Konzept des maschinellen Lernens

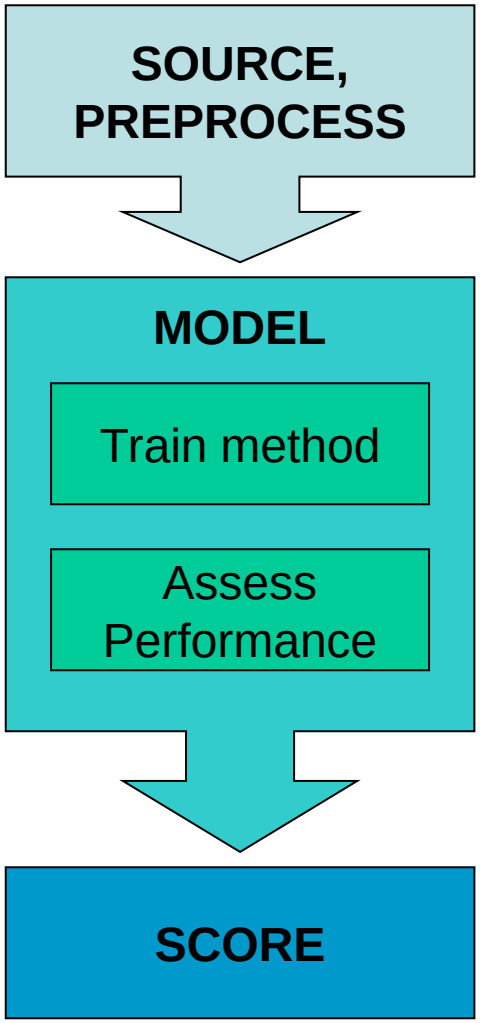
- Eine *Maschine* (Computerprogramm) hat aus der Erfahrung E für eine Aufgabe T gelernt, wenn T nach Erlernen von E hinsichtlich einer Performance P besser bewältigt wird:

Performance(Task|Experience) > Performance(Task)

- Das Ziel des *Lernenden* (=Algorithmus) ist es, aus Informationen zu *Beispielen* oder Instanzen (=Fälle), die durch *Features* bzw. Attribute (=Größen) beschrieben sind, ein reales Phänomen anhand einer *Hypothese* (=Modell) näherungsweise zu erklären.

© FH Technikum Wien

Workflow für prädiktive Methoden



- Datenextraktion aus verschiedenen Quellen
- Datensäuberung (Fehler, Fehlwerte)
- Standardisieren der Größen (=gleiche Skala)

Auswahl eines geeigneten Verfahrens:
- Trainieren des Modells mit (Teil der) Daten
- Parameter-Tuning, wenn möglich

- Beurteilung der Performance des trainierten Modells auf Basis eines unabhängigen Datensatzes
- Möglicher Vergleich mit einem anderen Verfahren, Auswahl des besseren Modells

- Einsatz des trainierten Modells, um Vorhersagen für neue Daten zu erstellen (z.B. Segmentieren von neuen Kunden)

Definition der Datenpunkte

- Wichtig für alle Weitere: in allen Machine Learning-Verfahren (einschließlich neuronaler Netze, siehe letztes Modul!) werden Daten als **n-dimensionale** Vektoren beschrieben
- Diese nennt man **Merkmals- bzw. Feature-Vektoren**
- Teil des **Preprocessings** (siehe oben) ist es, möglicht treffsichere Features zu finden, nämlich:
  - Für die Anwendung relevante Informationen codieren
  - Nicht zu viele/ wenige Dimensionen, Keine redundaten/ irrelevante Werte
- Beispiele für Anwendungen und mögliche Merkmale:
  - Anwendung „Filme nach Genre klassifizieren“: beteiligte DarstellerInnen, RegisseurIn, Länge, Produktionsländer, etc.
  - Anwendung „Handelsklasse von Äpfel vorhersagen“ (siehe später): Größe, Farbe
  - Anwendung „Bilder in Kategorien einteilen“: Farbwerte der einzelnen Pixel (theoretisch, in der Praxis → letztes Modul)

Typische Machine Learning Aufgaben

Überwachtes Lernen: Prozess der Erstellung eines Modells auf Basis eines definierten Lerndatensatz

- ***Klassifikation:*** Label sind *kategorial* (binär oder mehrere Klassen)
  Beispiel: Genre von Filmen anhand von Mitwirkenden vorhersagen
- ***Regression:*** ähnlich, aber Labels sind *kontinuierliche Werte*
  Beispiel: Vorhersage von Verkaufszahlen

Unüberwachtes Lernen: Erstellen eines Modells ohne Referenzdatensatzes und mit dem Ziel, Muster zu finden

- ***Clustering:*** Identifikation von ähnlichen Objekten
  Beispiel: Bilder zu Alben zusammenfassen
- ***Assoziationsregeln:*** Finden von zusammengehörigen Objekten
  Beispiel: Hinweise der Art „Leute die dies kauften, kauften auch das“

© FH Technikum Wien

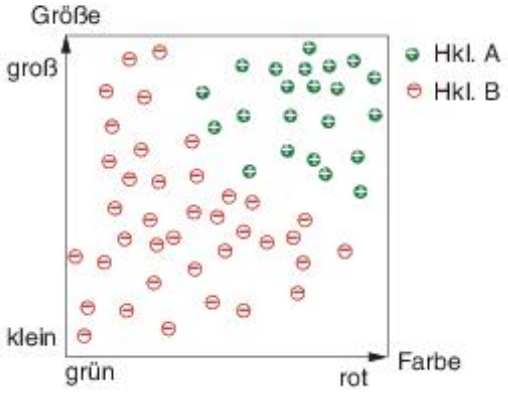
Beispiel für Klassifikation

Klassifikationsaufgabe:

Äpfel basierend auf den **Merkmalen (Features)** **Größe und Farbe** in Handelsklasse A und B sortieren

Größe [cm]	**8**	**8**	**6**	**3**	**...**
Farbe	**0,1**	**0,3**	**0,9**	**0,8**	**...**
Handelsklasse	B	A	A	B	...





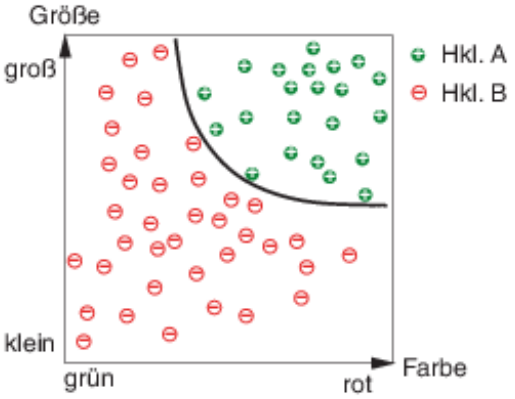
© FH Technikum Wien

Beispiel für Klassifikation

Ein System, das Instanzen basierend auf Merkmalsvektoren automatisch in die Klassen einteilt, nennt man ***Classifier***

Größe [cm]	**8**	**8**	**6**	**3**	**...**
Farbe	**0,1**	**0,3**	**0,9**	**0,8**	**...**
Handelsklasse	B	A	A	B	...





© FH Technikum Wien

Maschinelles Lernen

Man unterscheidet:

- Überwachtes (supervised) Lernen:
  Das System hat eine große Menge an
  klassifizierten Trainingsdaten
  und lernt daraus, auch neue (bislang nicht klassifizierte) Instanzen richtig einzuordnen.
- Unüberwachtes (unsupervised) Lernen:
  Es gibt **keine** Trainingsdaten. Das System erkennt Muster und Ähnlichkeiten in Daten, ohne Referenzwerte zu haben.

Beachte:

Man kann meist nicht frei auswählen was man verwendet!
Es kommt auf die Anwendung an.

© FH Technikum Wien

2. Überwachtes Lernen

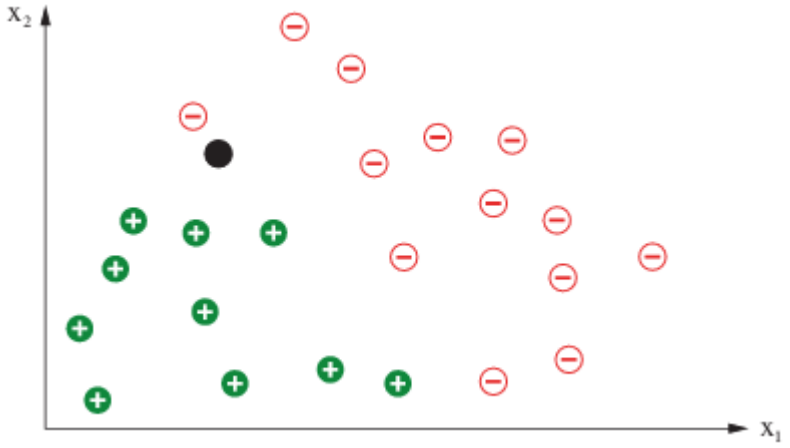
© FH Technikum Wien

Nearest-Neighbour (NN)

- Grundlegende Idee: Neuer Datenpunkt wird anhand des nächstgelegenen Punkts klassifiziert
- Wichtige Aufgabe: ***Ähnlichkeit*** identifizieren (geringer Abstand im Merkmalsraum)
- Etwa euklidische Norm:

$$d(\boldsymbol{x}, \boldsymbol{y}) = |\boldsymbol{x} - \boldsymbol{y}| = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

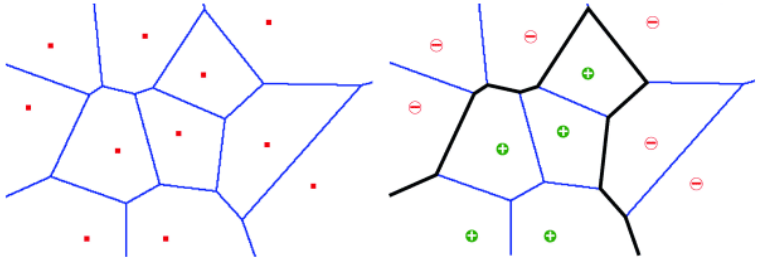
(es gibt jedoch noch viele andere Abstandsmaße!)



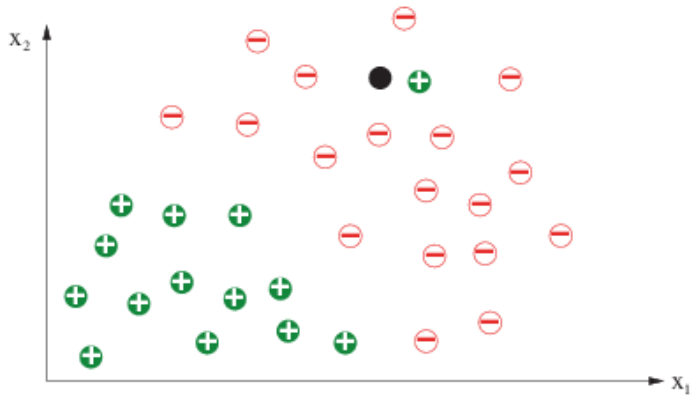
© FH Technikum Wien

Analyse von NN mittels Voronoi-Diagramm

- ***Voronoi-Diagramm:*** Jeder Datenpunkt ist von einem konvexen Polygon umgeben, welches das Gebiet um diesen definiert.
- NN-Klassifikation erlaubt Trennung von Klassen jenseits von Linien



- Problem ***Überanpassung*** (Overfitting): liegt in der Nähe eines neuen Punktes ein ‚falscher' Nachbar, können neue Daten falsch klassifiziert werden (Fehlanpassung an zufällige Fehler/Rauschen)

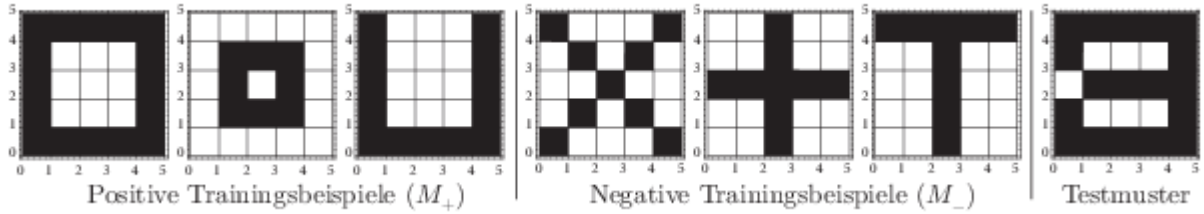


© FH Technikum Wien

k-Nearest Neighbour (kNN)

- NN problematisch wenn der nächste Nachbarn ein Ausreißer ist
- → Mehrheitsentscheid unter den *k nächsten Nachbarn*

- Beispiel: Mustererkennung: binäre Muster mit 5x5 Pixel



  - Berechne Distanz zu allen Trainingsbeispielen

    Mögliches Distanzmaß:

    Hamming-Abstand zweier Bit-Vektoren
    (Anzahl der unterschiedlichen Bits → Bitflips)
  - Klasse der k nächsten ergibt Klasse der Eingabeinstanz

© FH Technikum Wien

Erweiterung (k)NN für mehrere Klassen

- (k)NN-Klassifikation lässt sich problemlos auch ***auf mehr als zwei Klassen*** anwenden
  → Klasse des neuen Merkmalsvektors wird durch die Klasse des nächsten bzw. der k nächsten Nachbarn bestimmt

- Wird die Zahl der Klassen zu groß, ist es meist nicht mehr sinnvoll, Klassifikationsverfahren einzusetzen
  - Zahl der benötigen Trainingsdaten wächst stark
  - Gefahr von Overfitting
  - Information geht u.U. verloren

  → Bei numerischen „Klassen“ könnte Regression möglich sein

© FH Technikum Wien

Lernen von Entscheidungsbäumen

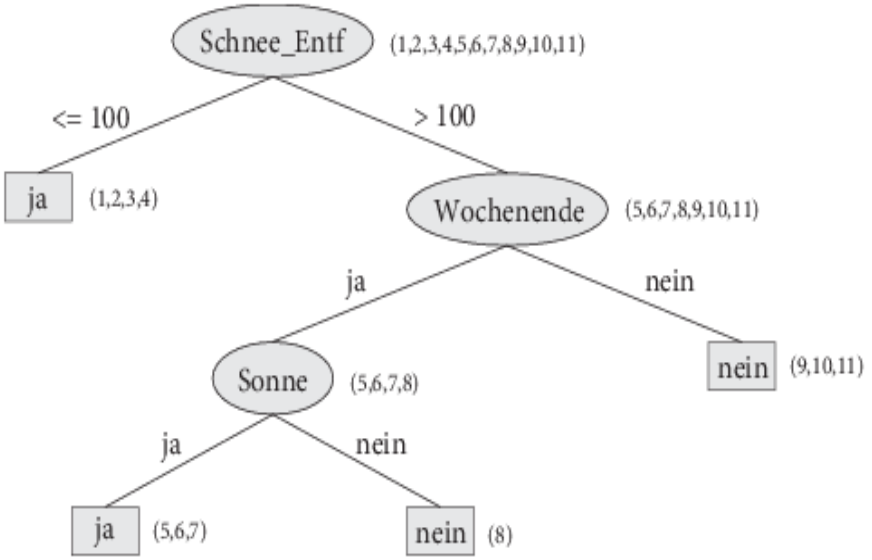
- ***Entscheidungsbäume:*** für die Praxis enorm wichtiges Verfahren (effizient und doch einfach)
  → auch sehr anschaulich, durch Mensch leicht zu verstehen, interpretieren und kontrollieren!
- Beispiel: Entscheidungsbaum für Skifahrer

Variable	Werte	Beschreibung
Skifahren (Zielvariable)	ja, nein	Fahre ich los in das nächstgelegene Skigebiet mit ausreichend Schnee?
Sonne (Merkmal)	ja, nein	Sonne scheint heute?
Schnee_Entf (Merkmal)	≤ 100, > 100	Entfernung des nächsten Skigebiets mit guten Schneeverhältnissen (über/unter 100 km)
Wochenende (Merkmal)	ja, nein	Ist heute Wochenende?

© FH Technikum Wien

Klassifikationsbaum: Definition, Beispiel

- Ein ***Klassifikationsbaum*** repräsentiert die ***Regeln***, um Antwortklassen von den gegebenen Attributen anzuleiten
- *Beispiel:* Wenn ein Schigebiet weniger als 100km entfernt ist, dann gehen wir heute Schifahren.



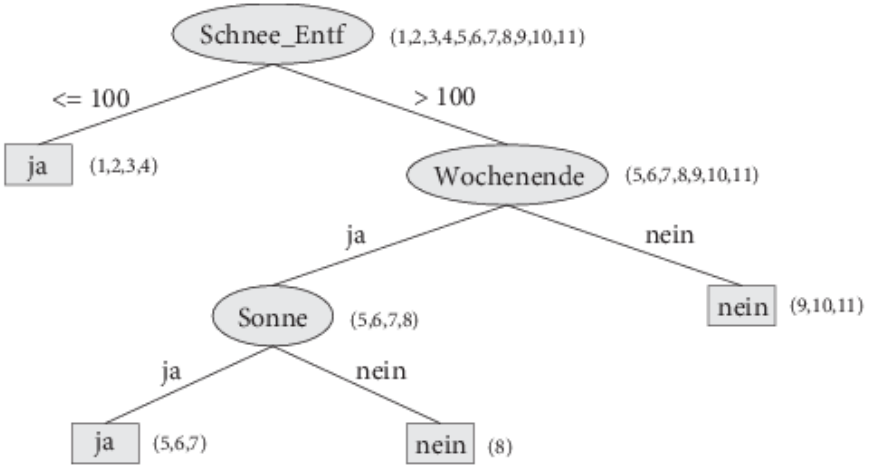
- An jedem neuen Tag kann die Entscheidung zum Schifahren auf Basis des Klassifikationsbaums abgeleitet werden – die erreichte Knoten bestimmt die Klasse.

© FH Technikum Wien

Entscheidungsbaum: Definition, Beispiel

Definition:

- Baum, dessen innere Knoten Merkmale (Attribute) repräsentieren
- Jede Kante steht für Attributwert
- An jedem Blattknoten ist ein Klassenwert angegeben



Tag	*Schnee_Entf*	*Wochenende*	*Sonne*	*Skifahren*
1	≤ 100	ja	ja	ja
2	≤ 100	ja	ja	ja
3	≤ 100	ja	nein	ja
4	≤ 100	nein	ja	ja
5	> 100	ja	ja	ja
6	> 100	ja	ja	ja
7	> 100	ja	ja	nein
8	> 100	ja	nein	nein
9	> 100	nein	ja	nein
10	> 100	nein	ja	nein
11	> 100	nein	nein	nein

→ Daten zum Aufbau des Beispiel-baumes (1 Eintrag pro Tag)

→ Manchmal wegen widersprüchlicher Daten (z.B. [6] und [7]) nicht in allen Fällen korrekte Klassifikation möglich

© FH Technikum Wien

Lernen von Entscheidungsbäumen

- Einschränkung auf Attribute mit endlich vielen Werten
- Brute Force-Ansatz: alle Bäume erzeugen und die Zahl der Fehlklassifikationen auf den Daten berechnen
  → minimale Fehlerzahl, aber zu hohe Rechenzeit
- Heuristische Algorithmus:
  Baum rekursiv (angefangen von der Wurzel) aufbauen:
  - In jedem Schritt wird nach dem *Attribut mit höchstem Maß an Unreinheit* gesplittet (z.B. Schnee_Entf <= bzw. > 100)
  - Attribute hat „hohe Unreinheit“ wenn nach dem Split im linken und rechten Teilbaum möglichst viele Instanzen dieselbe Klasse haben
- Genaue Formalisierung basiert auf mathematischer Definition von „Unreinheit“ (z.B. Entropie, Gini-Koeffizient)

© FH Technikum Wien

Pruning – der Baumschnitt

- ***Ockhams Rasiermesser*** (Occam‘s Razor):
  Bei gleichwertigen Theorien die einfachere bevorzugen

- Umgelegt auf Entscheidungsbäume:
  Existieren mehrere mit gleich guter (oder ähnlicher) Fehlerrate gibt, wähle den einfacheren (kleineren)

- Ziel eines Verfahrens:
  Erzeugen eines möglichst kleinen/einfachen Entscheidungsbaums → einfachere Modelle generalisieren besser auf neue Daten übertragbar und sind performanter

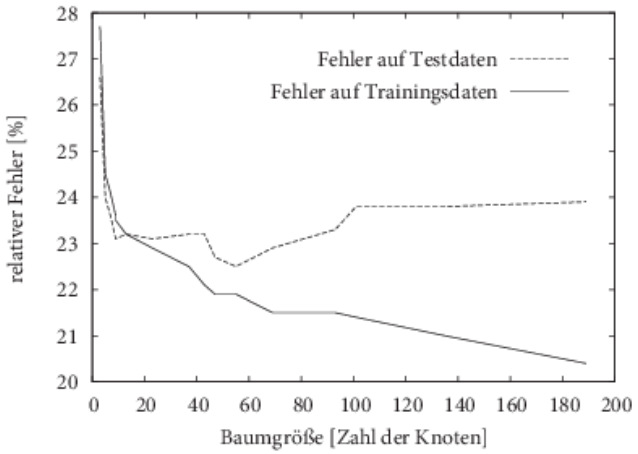
© FH Technikum Wien

Qualitätsbetrachtungen

Fehlerrate (Qualitätsmaß für den Baum): Erstellen mit Trainingsdaten (z.B. 70% der Daten)

→ Validieren mit Testdaten

→ Fehler auf Trainingsdaten nimmt monoton ab mit Größe des Baumes, nicht aber unbedingt auf den Testdaten



→ Beispiel: ab 55 Knoten steigt Fehlerrate wieder! (wie beim NN-Verfahren heißt der Effekt ***Overfitting*** bzw. Überanpassung)

→ Mögliche Lösung: Abbruch des Lernens bei 55 Knoten

© FH Technikum Wien

Gütekriterien von (supervised) ML-Verfahren

- Wie beurteilt man die Qualität eines gelernten Modells?
- Überblick über gängige Metriken:

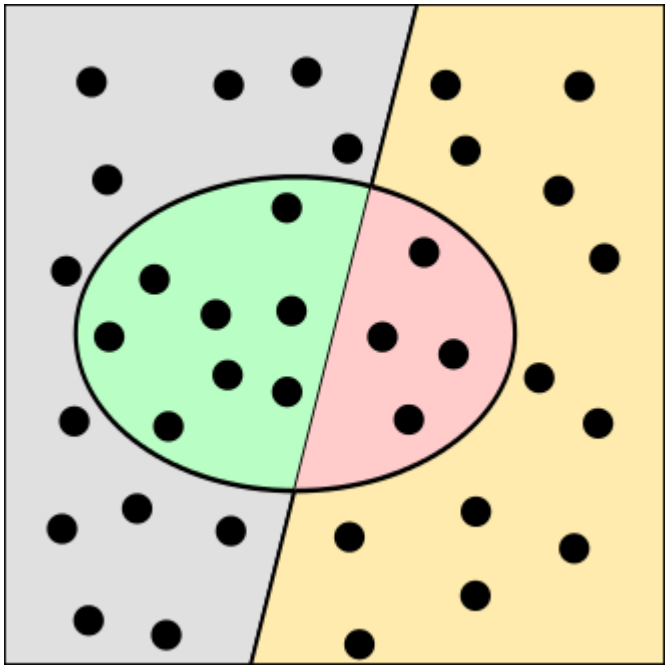
		True condition			
	Total population	Condition positive	Condition negative	Prevalence = $\frac{\Sigma \text{ Condition positive}}{\Sigma \text{ Total population}}$	Accuracy (ACC) = $\frac{\Sigma \text{ True positive} + \Sigma \text{ True negative}}{\Sigma \text{ Total population}}$
Predicted condition	Predicted condition positive	**True positive**, Power	**False positive**, Type I error	Positive predictive value (PPV), Precision = $\frac{\Sigma \text{ True positive}}{\Sigma \text{ Predicted condition positive}}$	False discovery rate (FDR) = $\frac{\Sigma \text{ False positive}}{\Sigma \text{ Predicted condition positive}}$
	Predicted condition negative	**False negative**, Type II error	**True negative**	False omission rate (FOR) = $\frac{\Sigma \text{ False negative}}{\Sigma \text{ Predicted condition negative}}$	Negative predictive value (NPV) = $\frac{\Sigma \text{ True negative}}{\Sigma \text{ Predicted condition negative}}$
		True positive rate (TPR), Recall, Sensitivity, probability of detection = $\frac{\Sigma \text{ True positive}}{\Sigma \text{ Condition positive}}$	False positive rate (FPR), Fall-out, probability of false alarm = $\frac{\Sigma \text{ False positive}}{\Sigma \text{ Condition negative}}$	Positive likelihood ratio (LR+) = $\frac{\text{TPR}}{\text{FPR}}$	Diagnostic odds ratio (DOR) = $\frac{\text{LR+}}{\text{LR−}}$
		False negative rate (FNR), Miss rate = $\frac{\Sigma \text{ False negative}}{\Sigma \text{ Condition positive}}$	Specificity (SPC), Selectivity, True negative rate (TNR) = $\frac{\Sigma \text{ True negative}}{\Sigma \text{ Condition negative}}$	Negative likelihood ratio (LR−) = $\frac{\text{FNR}}{\text{TNR}}$	

F_1 score = $\frac{2}{\frac{1}{\text{Recall}} + \frac{1}{\text{Precision}}}$

© FH Technikum Wien

Wahrheitsmatrix (Confusion Matrix)

- Klassifikationsbeispiel: Modell zur Einordnung von kranken (links) und gesunden (rechts) Menschen
- Grün: korrekt bewertete Fälle, rot: falsch bewertete Fälle



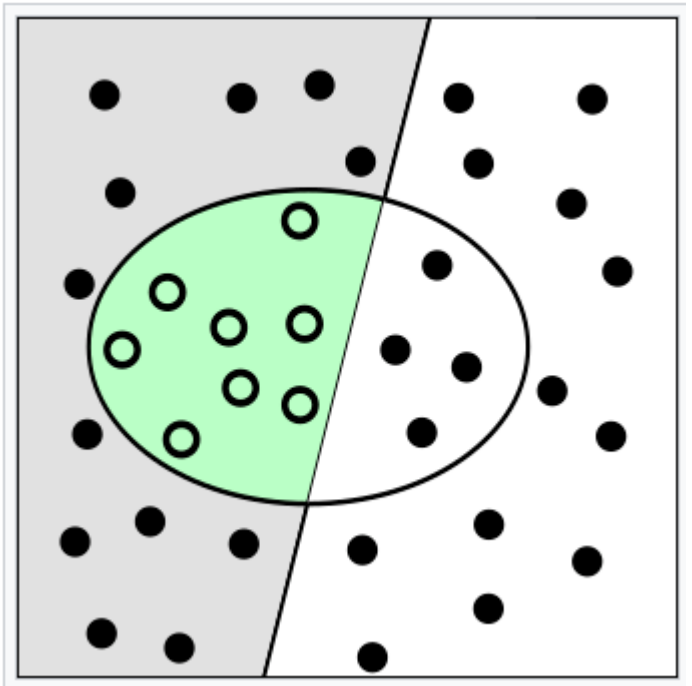
Wie schaut die WM für das Beispiel aus?

	Person ist krank (r_p+f_n)	**Person ist gesund (f_p+r_n)**
Test positiv (r_p+f_p)	richtig positiv (r_p)	falsch positiv (f_p)
Test negativ (f_n+r_n)	falsch negativ (f_n)	richtig negativ (r_n)

© FH Technikum Wien

Sensitivität (true/positive)

- Auch Richtig-positiv-Rate, Empfindlichkeit oder Trefferquote (Recall)
- Anteil der korrekt als positiv klassifizierte Objekte an der Gesamtheit der tatsächlich positiven Objekte



$$P(\text{positives Testergebnis}|\text{tatsächlich krank}) = \frac{r_p}{r_p + f_n}$$

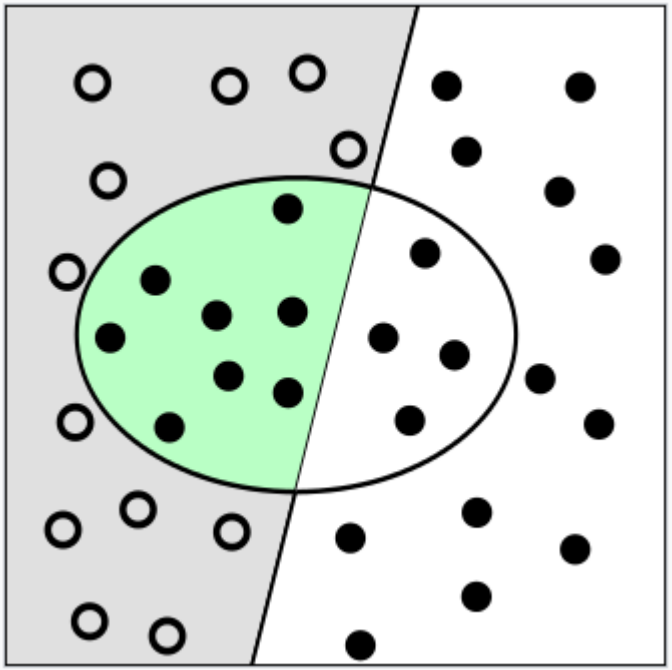
Wie groß ist die Sensitivität für das Beispiel?

© FH Technikum Wien

Falsch-negativ-Rate

- Anteil der fälschlich als negativ klassifizierten Objekte an der Gesamtheit der tatsächlich positiven Objekte an

$$P(\text{negatives Testergebnis}|\text{tatsächlich krank}) = \frac{f_n}{r_p + f_n}$$

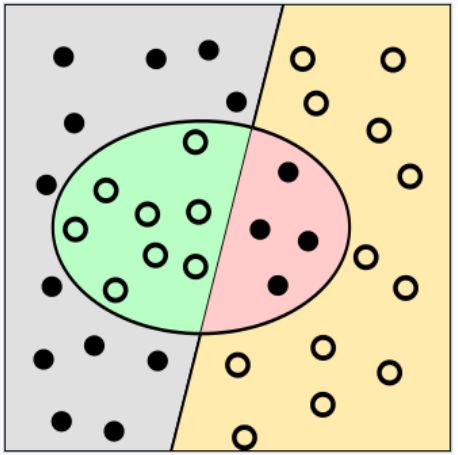


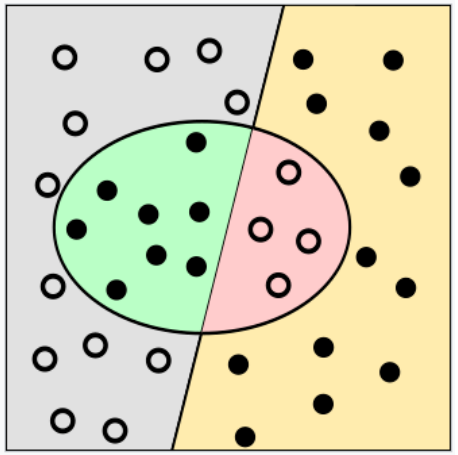
Was schaut diese Rate für das Beispiel aus?

© FH Technikum Wien

Korrekt- und Falschklassifikationsrate

- ***Klassifikationsrate*** (Treffergenauigkeit, ***Accuracy***): Anteil aller Objekte, die korrekt klassifiziert wurden (=Haupt-diagonale der Wahrheitsmatrix zur Gesamtheit)
- ***Klassifikationsfehler:*** falsch klassifizierte Objekte zur Gesamtheit (außerhalb der Hauptdiagonale)





$$P(\text{richtig klassifiziert}) = \frac{\text{richtig klassifiziert}}{\text{alle Fälle}} = \frac{r_p + r_n}{r_p + f_p + r_n + f_n}$$

$$P(\text{falsch klassifiziert}) = \frac{\text{falsch klassifiziert}}{\text{alle Fälle}} = \frac{f_p + f_n}{r_p + f_p + r_n + f_n}$$

© FH Technikum Wien

3. Unüberwachtes Lernen

© FH Technikum Wien

Clustering

- Erstes Beispiel für unüberwachtes maschinelles Lernen (→Trainingsdaten sind nicht gelabelt/klassifiziert)
- Ziel: Finden von Strukturen in Trainingsdaten
- In Cluster ist der Abstand benachbarter Punkte kleiner als der Abstand zwischen Punkten verschiedener Cluster
  → geeignetes Abstandsmaß festlegen



© FH Technikum Wien

Clustering: k-Means

- Wenn die Zahl der Cluster vorab bekannt ist → k-Means Verfahren (k Cluster mit den Mittelpunkten $\mu_1, \ldots, \mu_k$)
- Vorgehensweise:
  1. Klassifikation aller Daten zum nächsten Clustermittelpunkt
  2. Neuberechnung der Clustermittelpunkte

K-MEANS$(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n, k)$

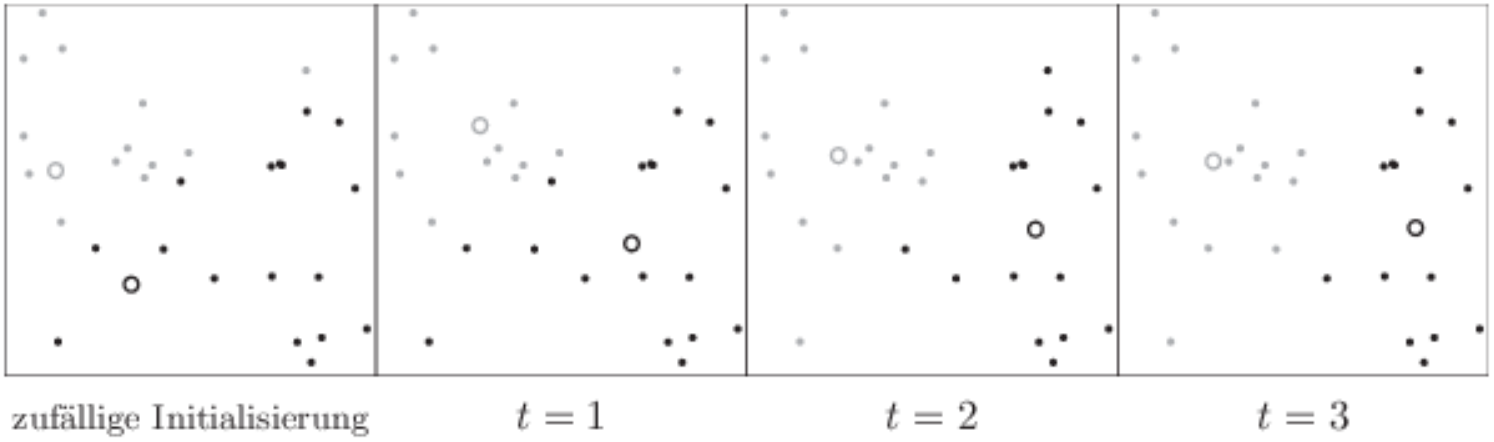
initialisiere $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k$ (z.B. zufällig)
Repeat
    Klassifiziere $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$ zum jeweils nächsten $\boldsymbol{\mu}_i$
    Berechne $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k$ neu
Until keine Änderung in $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k$
Return $(\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k)$

- Berechnung der Clustermittelpunkte: $\boldsymbol{\mu} = \frac{1}{l} \sum_{i=1}^{l} \boldsymbol{x}_i$

© FH Technikum Wien

Clustering: k-Means mit k=2

- Nach drei Iterationen stabilisieren sich die anfangs zufällig gewählten Klassenzentren (n=30 Datenpunkte):



- Keine Konvergenzgarantie, Verfahren konvergiert aber meistens recht schnell (d.h. Zahl der Iterationen viel kleiner als Zahl der Datenpunkte); Komplexität: ***O(ndkt)*** mit n=Punkte, d=Dimension, t=Zahl der Iterationen

© FH Technikum Wien

Bewertung eines Clusterings

- Wir wollen am Ende wissen wir gut oder schlecht ein Clustering ist.

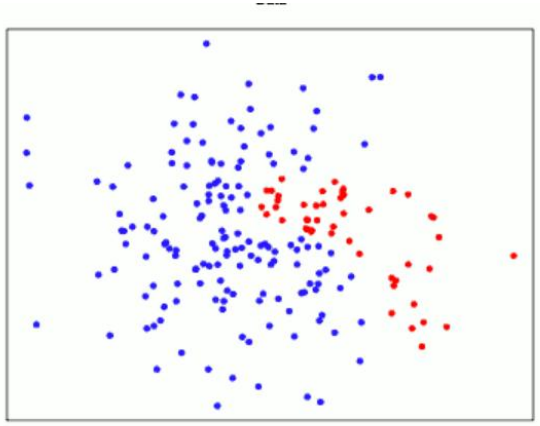
- Dazu braucht es ein Gütekriterium, und eines davon ist:
  Silhouette-Width-Kriterium:

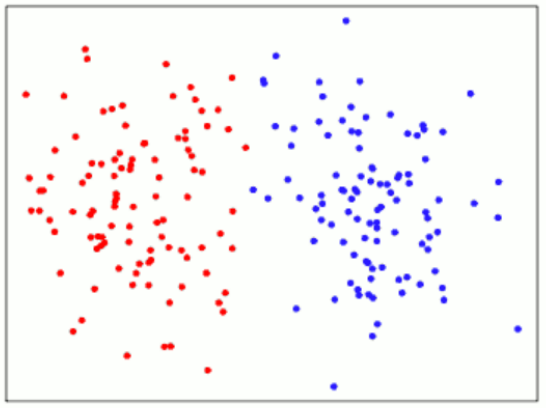
  Idee: Datenpunkte sollen möglichst gut im eigenen Cluster, und möglichst weit weg von fremden Clustern liegen.

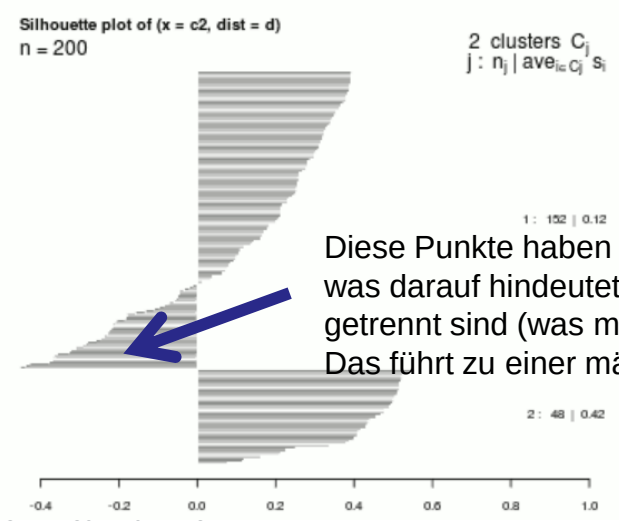
  (Kann man mathematisch genau definieren, für uns aber nicht relevant.)

© FH Technikum Wien

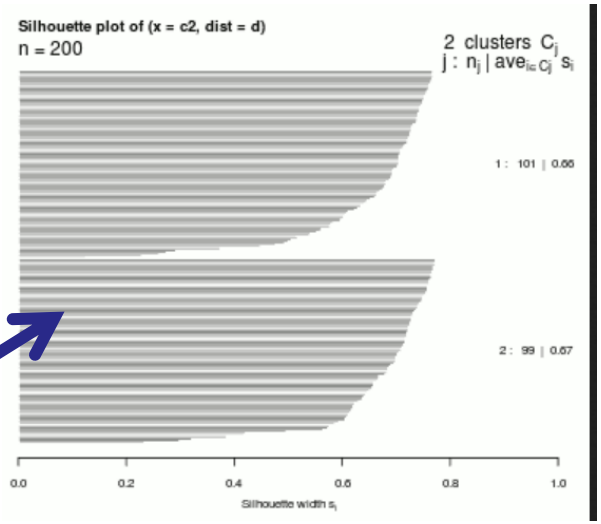
Silhouette-Width







Diese Punkte haben schlechte Werte, was darauf hindeutet dass die Cluster schlecht getrennt sind (was man in der Grafik auch sieht). Das führt zu einer mäßigen Gesamtbewertung.



Hier haben alle Punkte recht gute Werte → bessere Gesamtbewertung.

OMRk-Algorithmus

- Meistens weiß nicht im Voraus was eine gute Clusterzahl ist.
- Ausweg: k-Means für steigende Clusterzahlen (1 bis n) laufen lassen und bewerten wie gut das Clustering ist. Wähle dann das beste anhand Silhouette-Width.
- Das führt zum OMRk-Algorithmus (Ordered Multiple Runs of k-Mean):

$\text{OMRK}(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n, p, k_{max})$

$S^* = -\infty$
For $k = 2$ **To** k_{max}
  For i=1 **To** p
    Erzeuge eine zufällige Partition mit k Clustern (Initialisierung)
    Ermittle mit k-Means eine Partition P
    Bestimme S für P
    If $S > S^*$ **Then**
      $S^* = S$; $k^* = k$; $P^* = P$
Return (k^*, P^*)

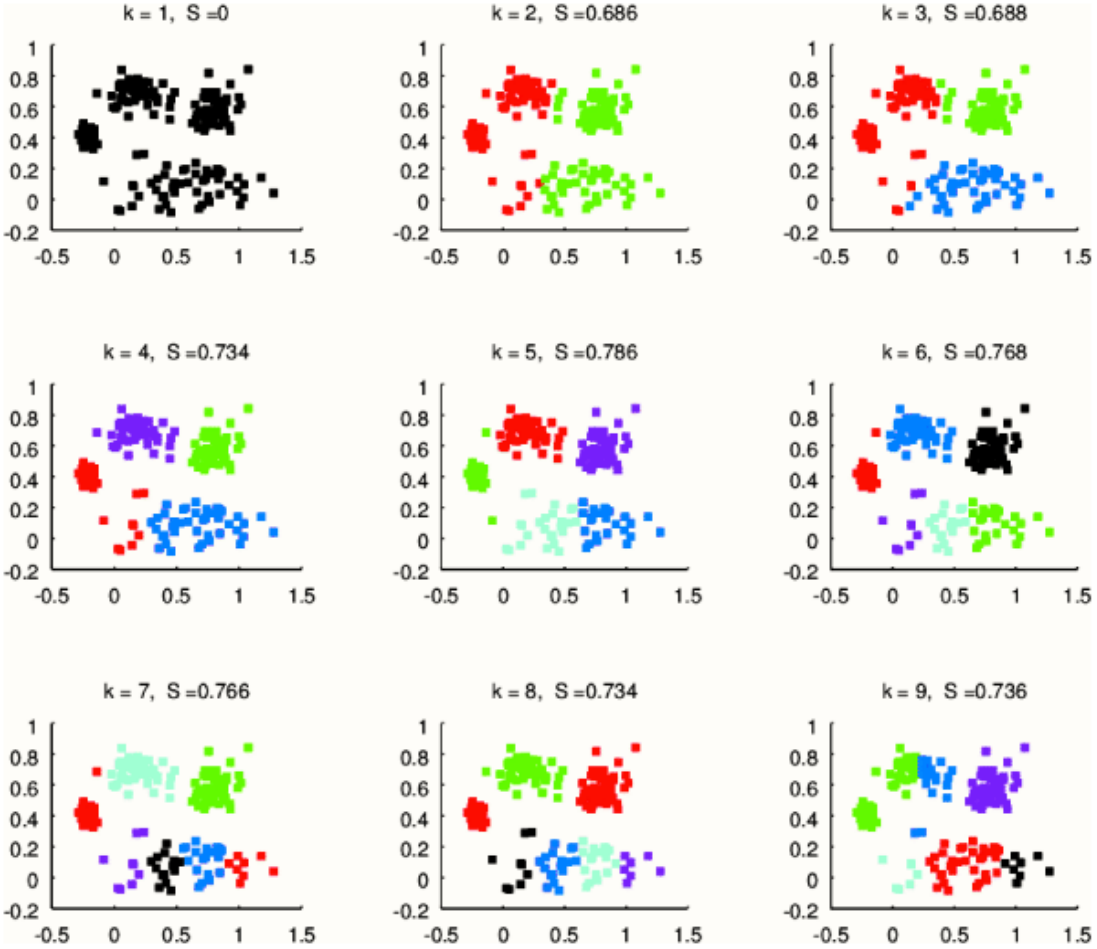
© FH Technikum Wien

Beispiel: OMRk-Algorithmus

Ergebnisse für k=2 bis 9

Bester Wert mit S=0.786 und k=5

Auge würde k=4 präferieren (höhere Dichte in Clustern)



© FH Technikum Wien

Eckpunkte des Moduls

© FH Technikum Wien

- Maschinelles Lernen umfasst ***überwachte und unüberwachte Lernverfahren*** sowie ***Lernen durch Verstärkung***. In der Regel kommen auch ***Data Mining*** Techniken zum Einsatz, um Semantik aus Rohdaten zu gewinnen.

- Klassifikation meint das ***Zusammenfassen von Objekten zu Klassen*** und wird typischerweise mittels ***überwachten Lernverfahren*** erstellt. Klassifikationsverfahren umfassen unter anderem Methoden wie ***k-Nearest-Neighbour*** oder ***Entscheidungsbäume***. Solche Modelle unterliegen ***Fehlklassifizierungen*** genauso wie dem Phänomen der ***Überanpassung***.

© FH Technikum Wien

- Bei ***unüberwachten Lernverfahren*** gibt es keine Label-Daten und auch kein Unterteilen des Datensatzes für Training und Validierung.

- Im Rahmen von Clustering ist entscheidend, ein gutes ***Abstandsmaß*** zu wählen. Mittels ***kMeans*** können Datenpunkte im n-dimensionalen Merkmalsraum in effizienter Weise geclustert werden.

- Die optimale Anzahl an Clustern kann entweder geschätzt oder durch ***Verfahren zur Bewertung eines Clusterings*** herausgefunden werden.

© FH Technikum Wien